

CI/CD hangs on tests — Flutter SDK Version Divergence

Technical solution of a one-day delay caused by a toolchain the team couldn't reproduce locally

SEVERITY

High — blocked CI green

DELAY

~1 development day

AREA

CI/CD · Flutter testing

ENVIRONMENT

GitHub Actions (Linux)

STATUS

Resolved · Prevention in place

TOOLCHAIN

Flutter pinned to 3.38.4

A fully green local Flutter suite kept failing — and hanging — on CI, even though no application code had changed. The root cause was a Flutter SDK version divergence between local development (3.38.4) and CI (latest stable), which changed how the test framework treated a pre-existing layout overflow. This document explains the symptom, the root cause, the investigation, the fix, and the prevention.

1 Executive Summary

The Court+ Flutter test suite of **123 tests** passed locally in ~4 seconds with `dart analyze lib test` fully clean. Yet the same commit repeatedly **hung and timed out on CI**, killing the Tests job at its 25-minute limit. Because the failure could not be reproduced locally, it initially looked like intermittent CI flakiness — but it was deterministic and version-specific.

Impact: three-plus push→run→fix cycles, two hung runs with 10-minute `TimeoutExceptions`, plus a false lead on the reporting job — together ≈ **one working day** spent on a defect that was **not in the application logic**.

! Key finding

“Works on my machine” was inverted: the code was fine, but the **toolchain differed** between local and CI, so a cosmetic layout overflow — silently tolerated by local Flutter — became a hard `FlutterError.onError` assertion on newer Flutter, which escalated into a test timeout.

2 Symptom

What we observed, and why it was confusing.

Observed behaviour

LOCAL Reproducible pass

dart analyze lib test → 0 errors. flutter test → **123 passed in ~4s**, every time, on every run.

CI Deterministic hang

Static Analysis job green, but Tests exceeded the 25-minute timeout and auto-killed. Same commit, same result — not a flake.

SIGNAL A pipeline, not an assertion

A test overrode `FlutterError.onError` but had unexpected additional errors followed by `TimeoutException` after 0:10:00: Test timed out after 10 minutes.

READINESS Blocking release

Red pipeline blocked merges; the same code worked in production-grade local verification, making the delay feel entirely “process,” not “product.”

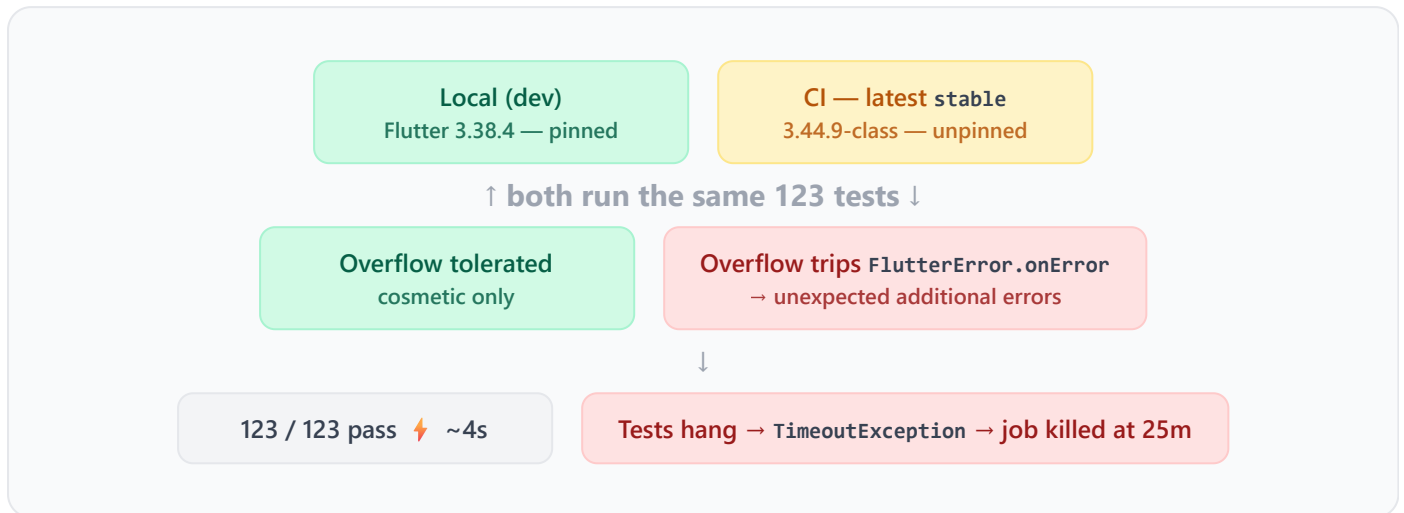
Run evidence

RUN	STATIC	TESTS	OUTCOME
31484304588	✓	TIMEOUT ~22M	Tests timed out (latest-stable Flutter)
31598596480	✓	TIMEOUT 25M	Tests hung; timeout + <code>FlutterError.onError</code>
31601086011	✓	1M36S	First green after pinning 3.38.4

3 Root Cause

Two independent factors compounded into the failure.

Version divergence



The failure chain

1 Pre-existing overflow

A settings widget had a latent layout overflow that older Flutter reported as a non-fatal console warning — never a test failure.

2 Toolchain change

Newer Flutter (3.44.9-class) tightened `flutter_test` error-tracking: the overflow now fired `FlutterError.onError` with unexpected additional errors, failing the test.

3 Hang, not fail-fast

In that configuration the failure manifested as a 10-minute `TimeoutException` inside the test, so the job consumed the entire CI window instead of failing quickly — hiding the true cause.

4 Why it cost a day

It was **impossible to reproduce locally** (local SDK 3.38.4 never failed), so each cycle required push → 10–25 min CI run → read logs → adapt. A separate reporting-job bug added a false lead.

4 Investigation & Timeline

How the team closed the loop from “flaky CI” to a version-pinning fix.

1 Reproduce & rule out the app

Confirmed `dart analyze` and `flutter test` are clean locally and that the failure is deterministic on CI — eliminating application logic as the cause.

2 Read the CI logs, not the code

Extracted the real signal: `FlutterError.onError` “unexpected additional errors” + `TimeoutException`, then identified

Note: CI pins flutter: latest ≠ local 3.38.4.

3 False lead: the report job

Initially suspected the CI report job. Found a separate, real bug there — it polled its `conclusion`, which is only set after the run ends (circular), so green runs stored as `in_progress`. Fixed via `needs.*.result` through `env`.

4 Pin the toolchain

Bumped all three CI jobs to `flutter-version: 3.38.4` to match local, and hardened widget tests to a small raster (`Size(400,800)` @ dpr 1.0) so they don’t trip the overflow on the Linux software renderer.

5 First fully green pipeline

Analyze ✓ 1m30s • Tests ✓ 1m43s • APK ✓ 7m35s • Report ✓ — and equivalence with local verification restored.

5 Resolution & Fix

The change set that closed the incident.

PINNING Explicit SDK version

CI now installs `flutter-version: 3.38.4` in the analyze, test, and build jobs — identical toolchain to local, so “passes locally” means “passes on CI.” No more floating to stable.

TESTS Robust widget tests

Widget tests run against a small raster (`Size(400,800)` @ dpr 1.0) instead of a 2.5M-pixel frame, avoiding the Linux software-renderer overflow and keeping the suite fast.

REPORT Real run status

The report job now derives its result from upstream job results via `needs.*.result` passed through `env` — fixing the circular conclusion read and `github-script@v7` `needs-global` removal.

GATE Green pipeline restored

Diet of the incident: a fully green, fast (~10 min) CI/CD pipeline with `status="success"` persisted to the dashboard.

6 Prevention & Lessons Learned

Actions that stop this class of incident from recurring.

1 Never float the toolchain

Pin the Flutter SDK (and by extension Node/Deno/Supabase CLI) with a specific version in CI, and keep local dev on the identical version. Record it in a machine-readable version file so the two can't silently diverge.

2 Make tests fail fast

Investigate and fix any `FlutterError.onError` that surfaces in the test harness; decide deliberately whether an overflow is a real defect or a tolerated cosmetic, rather than letting the toolchain decide for us.

3 Read the failure, not the guess

A 10-minute `TimeoutException` was the symptom; the cause was earlier in the log. Extract the first anomalous assertion before forming a theory, to avoid chasing the reporting job first.

4 Cache the outcome

Each resolution is anchored by a green run and a confirmed DB row, so "fixed" is evidence-based — a habit that removed most of this incident's one-day cost.